DECKBLATT FÜR HAUSARBEITEN
- Archäologisches Institut -

Durch Student/in auszufullen:

Abgabesemester: WiSe (15.03.) 2025/26

Vorname: Tin Name: Andrašec

Adresse (Str./PLZ/Ort): Annemarie-Renger-Straße 4a, 50129 Bergheim

E-Mail: tandraec@smail.uni-koeln.de

Matrikelnummer: 7419449 Fachsemester: 5

Kursnummer /-titel: 144360398 Agent-based modelling for archaeologists. From concept to application and publication

Modulkennung /-titel: 4436BMFAM1 Advanced Module Modulprufung: ja ☒ nein ☐

Name Prufer/in.: Andreas Angourakis,

Titel der Hausarbeit:

Modelling Middle Stone Age Foraging Patterns

Erklärung

Hiermit versichere ich, Andrašec, Tin
Name, Vorname

23.02.2000
geboren am

dass ich die vorliegende, an diese Erklärung angefugte Hausarbeit selbstständig und ohne die Benutzung anderer als der angegebenen Hilfsmittel angefertigt habe. Alle Stellen der Hausarbeit, die wörtlich oder sinngemäß aus veröffentlichten Schriften entnommen wurden, sind als solche unter Angabe der Quelle kenntlich gemacht.

Bergheim, 15.3.2026
Ort, Datum

Unterschrift

ARCHÄOLOGISCHES INSTITUT
DER UNIVERSITÄT ZU KÖLN

Kerpener Straße 30 / Eingang Weyertal
Postanschrift: Albertus-Magnus-Platz
50923 Köln

Bewertung der Hausarbeit

Titel der Hausarbeit: Modelling Middle Stone Age Foraging Patterns

Name der/des Studierenden: Tin Andrašec

Durch Prufer/in auszufullen:

Note: _____ Gesamtnote mit Referat: _____

Unterschrift Prufer/in: _____ Ort, Datum: _____

Kommentar: _____

Modelling Middle Stone Age Foraging Patterns

Introduction

Still Bay (SB) and Howiesons Poort (HP) are two lithic industries endemic to southern Africa, dated to approximately 72–59 ka, which have received a lot of scientific attention because of their association with some of the most earliest attestations of symbolic behaviour in southern Africa, as well as with bone tool production. Therefore, it has been argued that SB and HP industries reflect the first appearance of the “behavioural modernity” among early anatomically modern human populations.

It has been argued that both SB and HP industries show significant differences from the subsistence practices, mobility patterns and settlement systems of both the preceding and the succeeding periods of the Middle Stone Age of Africa. However, there are differences between both SB and HP subsistence strategies. Typical lithic artefacts for the SB industry include bifaces, which have been used as projectile weapons, and seem to be related to a pattern of frequent and/or long-distance residential moves. On the other hand, microlithic technology of the HP period seems to have been used for more specialised tool design.

McCall and Thomas (2012) note several differences in foraging behaviour of SB and HP populations. SB faunal data implies encounter-based hunting strategy focused on highly ranked prey animals, together with a high frequency of residential mobility and movement (McCall and Thomas 2012, 30).

On the other hand, HP foraging strategies includes longer-term occupation of residential camps, as well as the exploitation of diverse small game. However, it seems that long-range logistical trips also constituted an important part of the subsistence (McCall and Thomas 2012, 29).

Therefore, these two different foraging behaviours will be modelled using an agent-based model. The experiments will include placing agents representing camps and foragers in a simulated environment, and observing how well they thrive in it. The available resources will be represented with energy values. Although SB and HP communities focused on different resources, it will be assumed that during both periods harvestable energy was uniformly distributed across the landscape.

It would be expected that, if the change from SB to HP is due to the environmental reasons, the two foraging strategies will not have the same degree of success in the same environment.

Methods

This project was created to explore the question of the transition from the Still Bay (SB) to the Howiesons Poort (HP) lithic industry in Southern Africa. It is presumed that the differences in material cultures are caused by the differences in subsistence strategies of the two populations. Subsistence strategy typical of each lithic industry will be modelled, and both populations will be placed in simulated environments.

The experiment will model two different different types of foraging behaviour:

- SB – long-distance migration of camps, foraging parties venture further from the camp
- HP – short-distance migration of camps, foraging parties stay near the camp

The experiments will include placing agents representing camps and foragers in a simulated environment, and observing how well they thrive in it. The available resources will be represented with energy values. Although SB and HP communities focused on different resources, it will be assumed that during both periods harvestable energy was uniformly distributed across the landscape.

It would be expected that, if the change from SB to HP is due to the environmental reasons, the two foraging strategies will not have the same degree of success in the same environment.

Model Description

Purpose

The purpose of the model is to simulate the behaviour of Palaeolithic hunter-gatherer communities. The model creates a randomly generated landscape, with energy being randomly distributed across the landscape. Hunter-gatherer communities are represented by agents of breed “camps”. Each camp creates foraging parties, represented by the breed “forager”, which harvests energy from the environment and brings it to the camp. After the energy in the vicinity of the camp has been depleted, the camps relocate to another part of the landscape. As time progresses, camps and forager parties consume the gathered energy. In the case that all the energy reserves of a camp are exhausted, the camp perishes. If the camps are successful in harvesting energy, the population grows, and eventually the camps split, creating new communities. The model thus enables comparison of different foraging strategies.

Parameters and variables

The model was created in NetLogo development environment version 7.0.3. It requires the following input parameters:

Parameter	Type	Value Range	Description
camp-number	Integer	1 to 10	Number of camps at the start of the simulation run.
starting-population	Integer	10 to 100	Population of starting camps.
init-camp-energy	Integer	0 to 2000	Amount of energy stored by each starting camp.
max-penergy	Integer	0 to 100	Maximum value of the penergy variable of a patch.
init-forager-energy	Integer	0 to 100	Amount of energy stored by each starting forager.
resource-density	Integer	0 to 100	Density of energy in the environment. The chance that in a simulation run a patch will contain energy.
gather-rate	Integer	0 to 100	Amount of energy which a forager will harvest in a tick from a patch where it is located.
growth-rate	Floating point number	0 to 0.25	Amount by which the energy in harvested patches recovers each tick.
pop-growth-chance	Floating point number	0 to 0.1	Chance per tick that, given there is enough energy, the population of a camp will grow.
forage-step	Integer	1 to 5	Distance which a forager covers each tick.
forager-movement-cost	Integer	1 to 10	Amount of energy consumed by a forager when traversing a length of an unit of distance while foraging.
camp-movement-cost	Integer	1 to 100	The amount of energy consumed by a camp when traversing a length of an unit of distance while migrating
return-home-threshold	Integer	1 to 100	After harvesting this amount of energy from the environment, the forager returns back to its camp.
migration-step	Integer	1 to 50	Distance a camp covers during a single migration event.
pop-energy-consumption	Integer	1 to 10	Amount of energy a unit of population consumes each tick.
split-pop-pop-threshold	Integer	0 to 100	Population value at which, once reached, a camp will split into two camps.
split-pop-energy-threshold	Integer	0 to 4000	Stored energy required for splitting the population into two camps.

Table 1: List of input parameters used in the model.

The model has three agent (or, using NetLogo terminology, “turtle”) breeds: patches, camps, foragers, each breed possessing the following variables:

Breed	Variable	Type	Description
patches	penergy	Floating point number	The amount of energy available for harvest on a patch.
patches	pempty	Boolean	Indicates whether energy can be harvested from a patch.
camps	my-foragers	Agentset	Agentset containing all foragers belonging to a camp.
camps	camp-energy	Floating point number	Amount of energy stored in a camp.
camps	migrate-now	Boolean	Indicates whether the migration process should begin.
camps	population	Integer	Population of a camp.
foragers	my-home	Agent	Camp to which an agent belongs.
foragers	encounter-list	List	Energy values harvested form the last 10 visited patches.
foragers	encounter-rate	Floating point number	Average value of the elements in the encounter-list.
foragers	forager-energy	Integer	Energy stored by each forager.
foragers	return-home	Boolean	Indicates whether a forager start returning to his camp (home), ending the foraging process.
foragers	migrate-now	Boolean	Indicates whether the camp to which a forager belogns to will begin its migration process.

Table 2: List of variables which possesed by differed agent breeds.

The simulation landscape is represented by the NetLogo world, with maximum x coordinate of the patches being 50, and maximum y coordinate set at 40. The world wraps both horizontally and vertically.

Process overview and scheduling

The model contains the following procedures:

Procedures	Observer
setup	global
go	global
forage	foragers
gather	foragers
migrate-check	camps
split-population	camps
migrate	camps
camps-eat	camps
foragers-eat	foragers
Transfer-storage	foragers
increase-population-check	camps
update-display	patches
regrow-patches	patches

Table 3: List of procedures used in the model.

Setup

Before the start of a simulation run, the “setup” procedure (figure 1) has to be run. To start with, all variables are cleared using the “clear-all” procedure. Next, the generation of the simulation landscape begins. Patches are set as the observer, and each patch runs an if-else statement in which the value of a randomly generated number is compared to the previously defined “resource-density” parameter. If the randomly generated number is lower than the “resource-density”, energy is allocated to a patch: its “penergy” variable is set to the value of “max-penergy” parameter. Furthermore, “pempty” is set to “false”. This variable will later be used to evaluate whether the energy at the patch should be replenished. If that is not the case, “penergy” is set to 0, and “pempty”

is set to “true”. Afterwards, colour of the patch is set in such a way to reflect the amount of energy available on the patch.

Secondly, the procedure creates agents belonging to the breed “camps”. The number of camps which are thus created is specified by the “camp-number” parametre. The camps are then randomly distributed across the simulation landscape, and their starting population and stored energy are set up according to the value of “starting-population” and “init-camp-energy” parameters.

Then, for every 10 units of population, a camp hatches a forager. The foragers then set their “my-home” variable to include the camp which hatched them, after which they set their “return-home” and “migrate-now” variables to “false”. The starting encounter list is setup: it contains 10 values, each equalling twice the amount of “max-penergy”. “Encounter-rate” is set to the mean of the “encounter-list”. Consequently, “forager-energy” is set to the value of “init-forager-energy”.

Finally, the observer again switches to the camps, and each camp stores their newly created foragers in their my-foragers agentset variable. Then, the tick counter is reset.

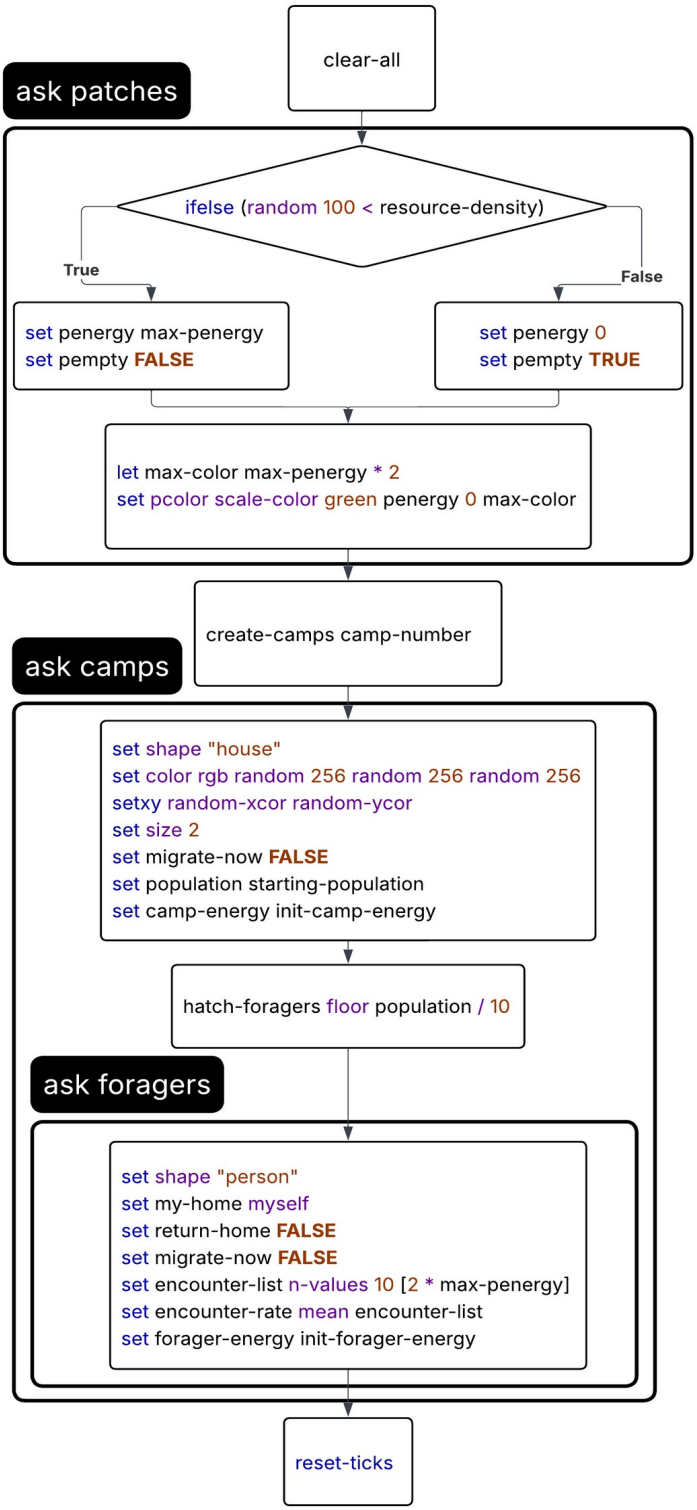

Figure 1: Flow chart of the setup procedure.

Go

At the start of the “go” procedure (figure 2), foragers are set as the observer. First, a forager runs the “foragers-eat”, procedure, thus reducing its “forager-energy” variable by the set amount. Afterwards, a forager is asked to check the value of its “return-home” and “migrate-now” variables. If either one of those variables is set to “true”, the forager runs the “travel-home” procedure, heading to its camp. Otherwise, the “forage” procedure is run, with the agent traversing the environment and harvesting the energy, if possible. This is repeated for all agents of the breed.

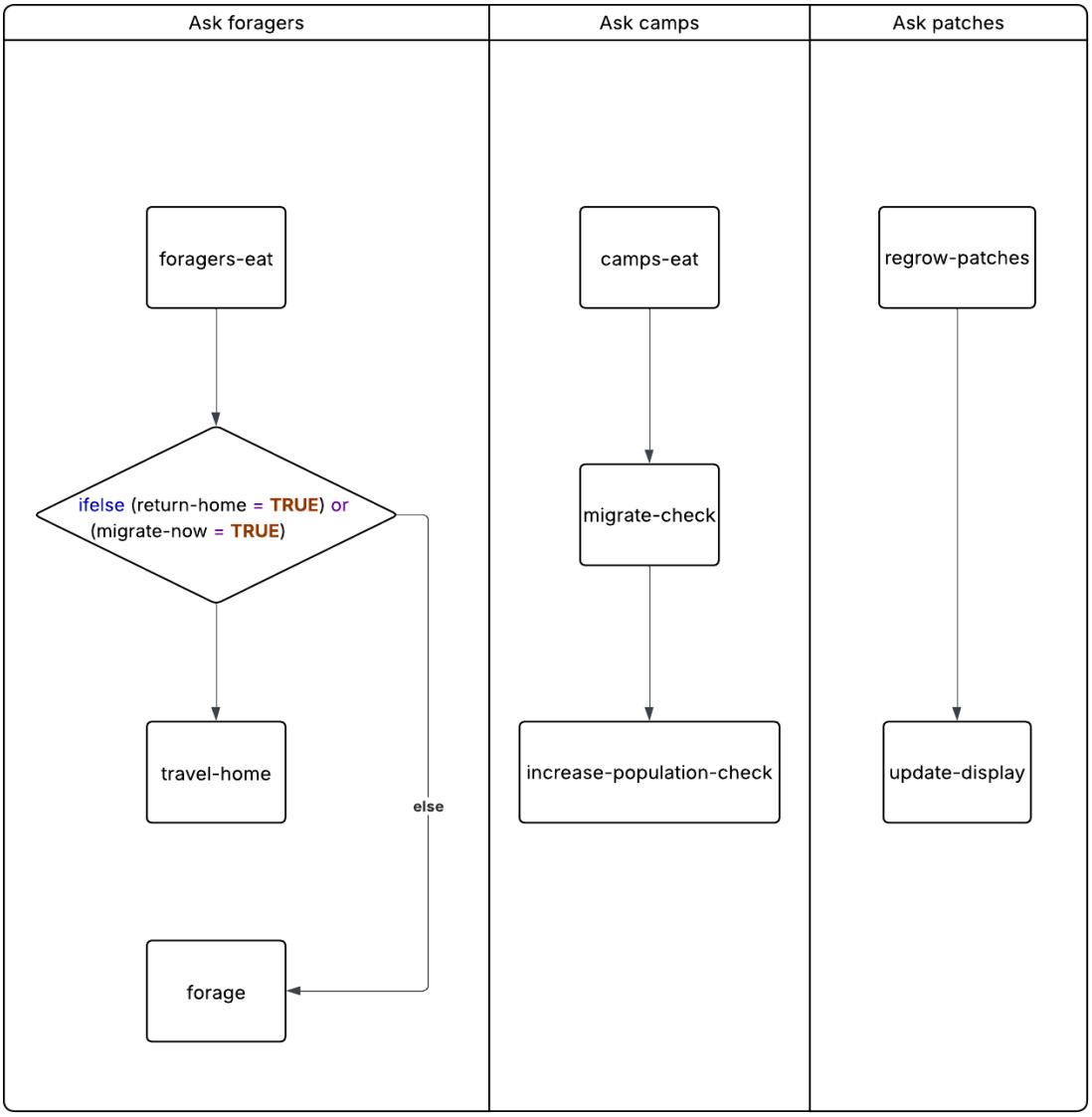

Figure 2: Flow chart of the go procedure.

Next, camps are set as the observer. First, a camp runs the “camps-eat” procedure, thus expending some energy as defined by the input parameters. After that, the camp runs the “migrate-check” procedure, evaluating whether the conditions for migration are met. Following that, the camp runs the “increase-population-check” procedure, ascertaining whether the population of the camp should be increased. This process is repeated for every instance of a camp.

Subsequently, patches are set as the observer. Initially, a patch runs the “regrow-patches” procedure, which increases the amount of energy in the patches which contain resources. Afterwards, the

“update-display” procedure is run, changing the colour of the patch according to the new value of its “penergy” value. This is again repeated for all patches.

Finally, the tick number is advanced by one.

Forage

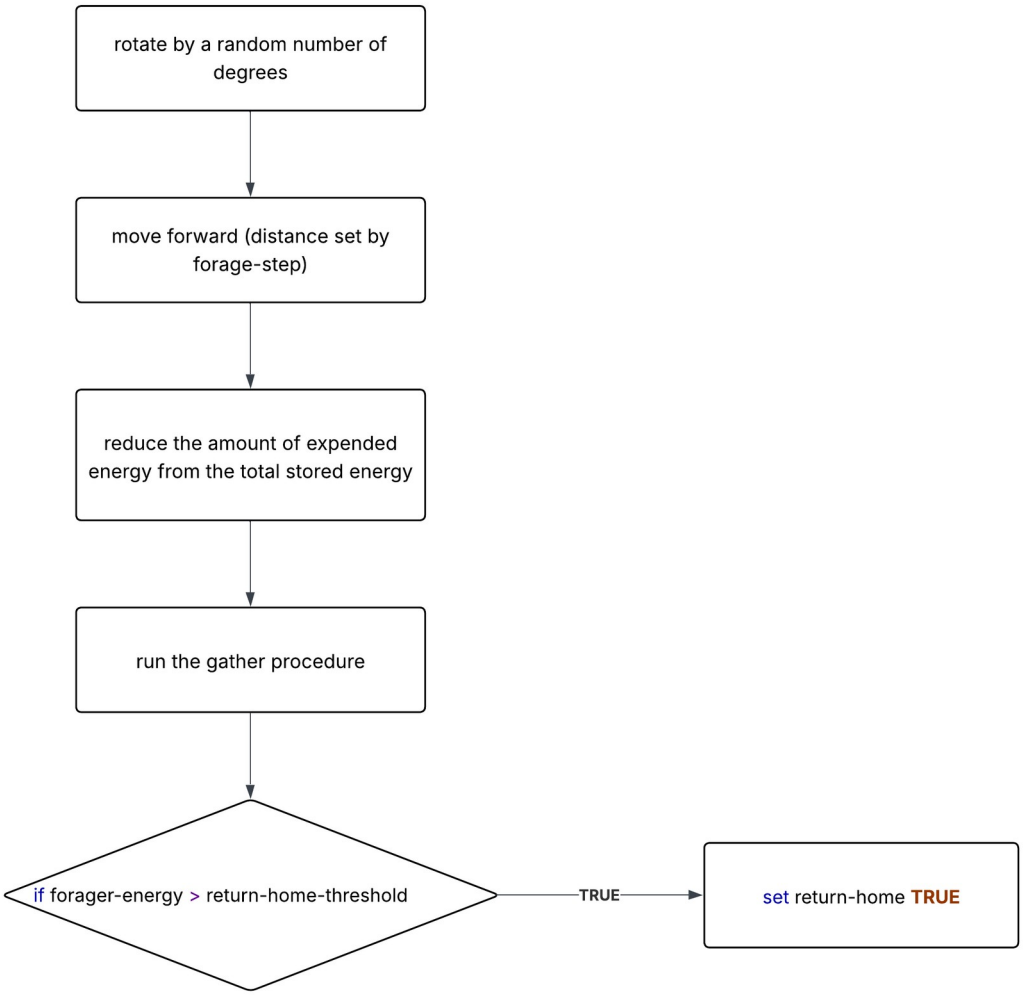

Figure 3: Flow chart of the forage procedure

The “forage” procedure (figure 3) is run with foragers are set as the observer. Initially, a forager is rotated by a random number of degrees. Next, it moves forward, the distance of this movement being set by the “forage-step” parameter. Thereafter, an amount of energy (as set by the starting parameters) is reduced from its energy storage. Next, it runs the gather procedure. Finally, it evaluates whether the value of its stored energy is higher than the “return-home-threshold” parameter. This being the case, it sets the value of the return-home variable to “true”.

Gather

The “gather” procedure (figure 4) is run with foragers set as the observer. In the code, it is only called within the “forage” procedure. To start with, two temporary variables are initiated: “patch-energy” and “gather-amount”. “Patch-energy” is set to the value of “penergy” variable of the patch where a forager is currently located. In the case that the value of “patch-energy” is greater than zero, a process where value is added to “gather-amount” is activated. Otherwise, the value of zero is

added to the forager's "encounter-list", placing it as the first item of the list. Next, the last item of the list is removed.

In the case that the value of "patch-energy" is greater than zero, another conditional statement is run. This time, the "patch-energy" variable is compared to the "gather-rate". If "gather-rate" is higher than "patch-energy", "gather-amount" is set to the value of "gather-rate". In other words, the forager does not gather all the available energy from the patch, but only the maximum amount he is allowed to gather by the initial parameters of the simulation run. Next, the patch where the forager is located ("patch-here") is set as the observer, and its "penergy" variable is decreased by the "gather-amount".

In the event that "patch-energy" is less than "gather-rate", the "gather-amount" is set to the value of "patch-energy." In simple terms, the forager collects all the energy which is available on the patch. Again, the "patch-here" is set as the observer, and its "penergy" is then set to zero.

After energy has been gathered, the "forager-energy" variable of the forager is increased by the gathered amount. Next, the forager's "encounter-list" is updated, as was described above.

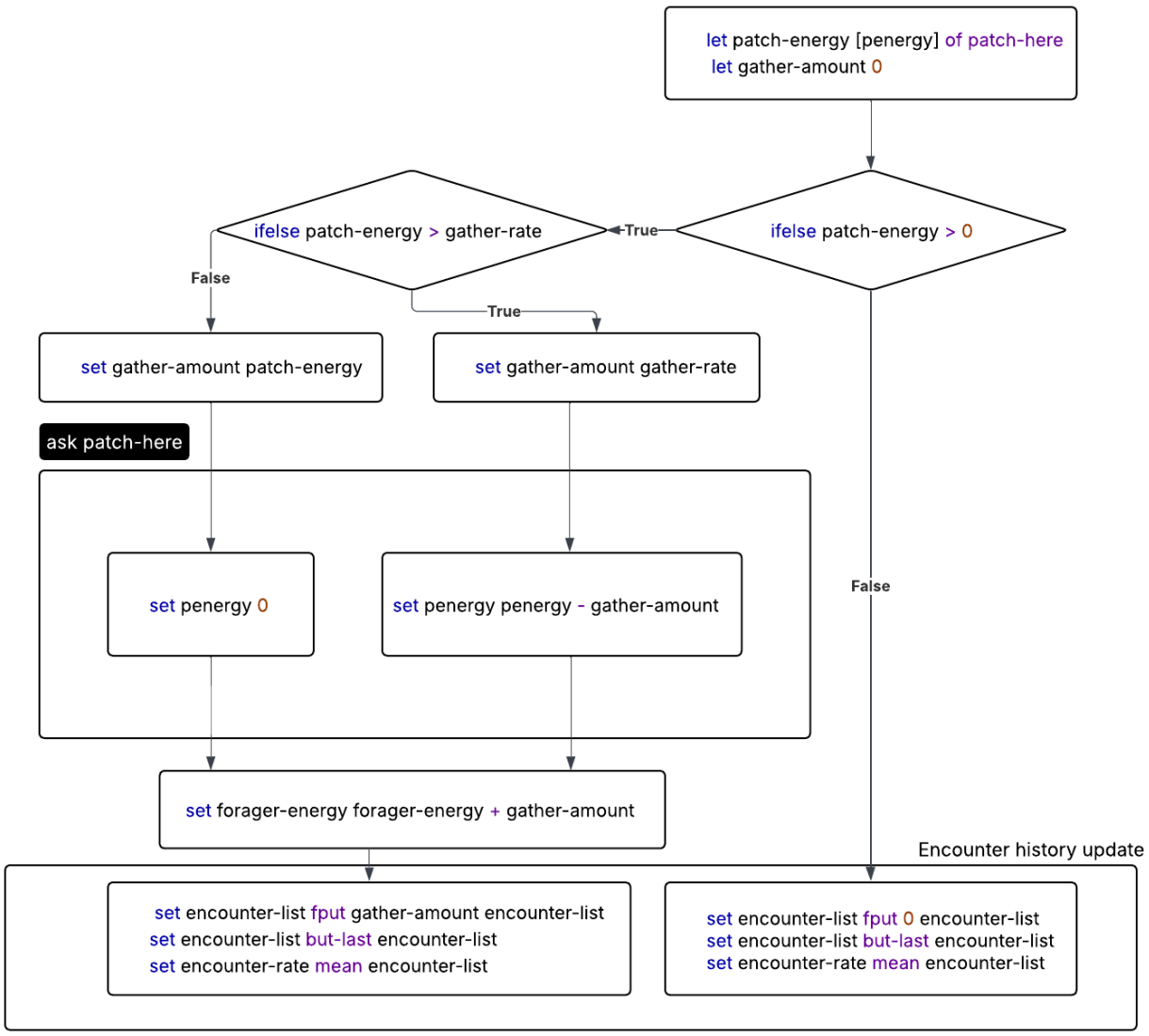

Figure 4: Flow chart of the gather procedure.

Migrate-check

The “migrate-check” procedure (figure 5) is run with camps as the observer. To start with, it evaluates whether the “migrate-now” variable of a camp is set to “true”. In the case that it is set to “true”, the camp will evaluate whether all the conditions for migration are satisfied.

Thereafter, a camp will check whether there are any agents in its “my-foragers” variable, i. e. it will check whether any of its foragers are still alive. In the case that there are no living foragers which belong to that camp, the “migrate procedure” will be run. However, before that, it will be evaluated whether the values of both population and stored energy exceed the threshold for splitting the camp into two. If that is the case, the “split-population” procedure will be run before the “migrate” procedure.

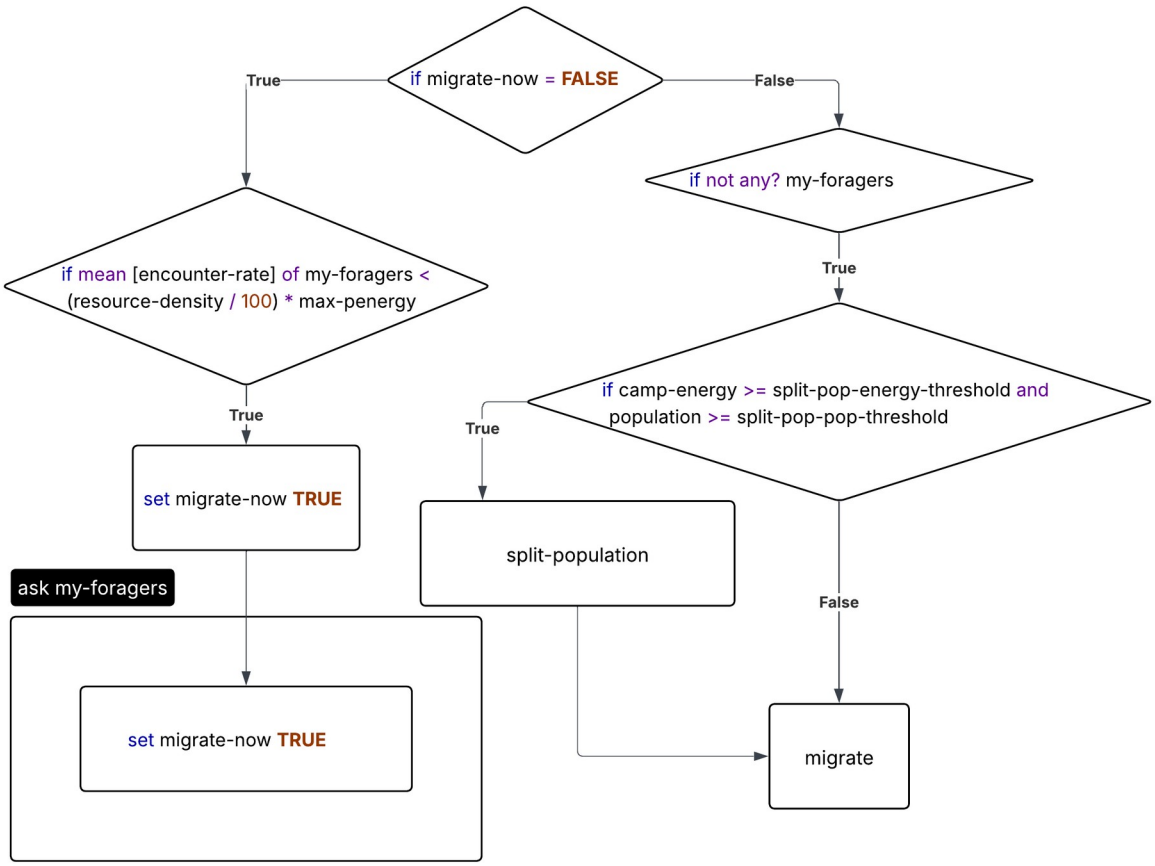

Figure 5: Flow chart of the “migrate-check” procedure.

Split-population

The “split-population” procedure (figure 6) is run within the “migrate-check” procedure. Therefore, a camp is an observer. The camp is asked to hatch another camp. By the definition of the “hatch-<breed>” function in NetLogo programming language, hatched agents inherit all variables from the observer agent. Next, the newly hatched camp sets the values of its population and energy variables to half of that of the original camp. After that, the original camp also halves its population and stored energy.

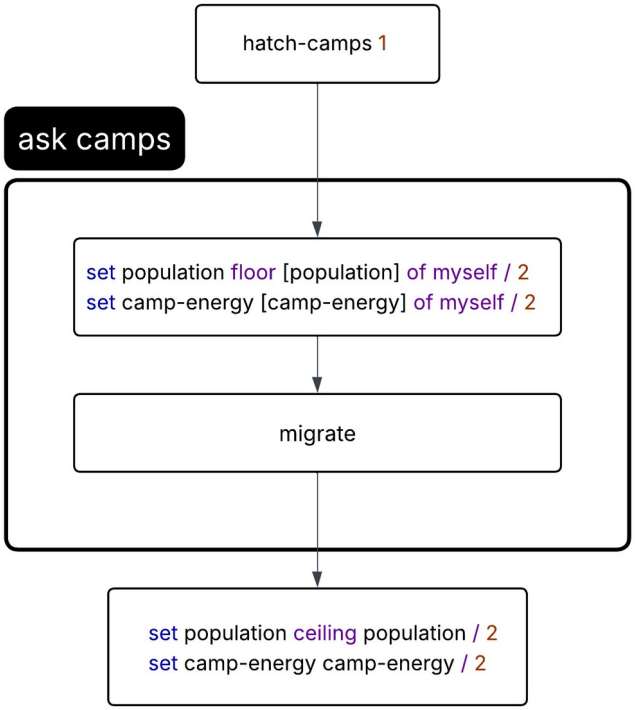

Figure 6: Flow chart of the split-population procedure.

Migrate

The “migrate” procedure (figure 7) is run within the “migrate-check” procedure, with camps being the observer. Firstly, the camp running the procedure rotates for a random number of degrees, and makes a step forward. The length of the step depends on the value of the “migration-step” parameter. Thereafter, a variable named “done” is initialised, which is used to control a while loop. The purpose of the loop is to ensure that the migrating camp migrates to an empty patch. If “done” equals “false”, the camp will evaluate whether there are any other camps present at its patch. If there is another camp present, the camp will move to one of the neighbouring patches. Otherwise, the value of “done” will be set to true, thus ending the while loop.

Following that, an amount of energy will be removed from the energy storage of the camp. This amount will depend on the length of the migration step, and the “camp-movement-cost” parameter, set at the beginning of the simulation run. Furthermore, the “migrate-now” variable of the camp will be set to “false”. Finally, depending on the value of its population variable, the camp will hatch a number of foragers, which will be accessible through its “my-foragers” agentset variable.

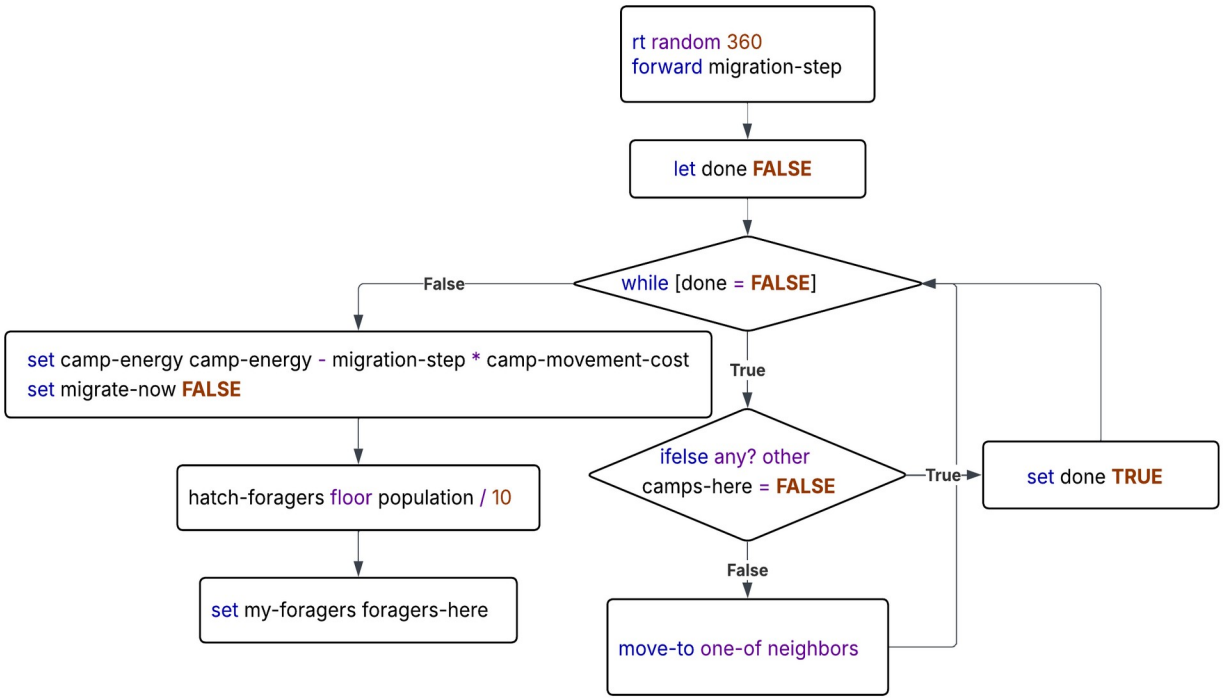

Figure 7: Flow chart of the "migrate" procedure.

Camps-eat

This procedure is called within the "go" procedure, with camps being the observer. It reduces the value of a camp's "camp-energy" variable by the multiple of "pop-energy-consumption" and the population of a camp. In other words, "pop-energy-consumption" parameter regulates how much energy an unit of population will consume in a tick. After that, the value of the camp's "camp-energy" variable is checked: if it is less or equal to zero, both the camp and it's foragers are asked to die.

Foragers-eat

This procedure is called within the "go" procedure, with foragers being the observer. It checks whether the "forager-energy" variable of a forager is less or equal to zero. If that is the case, the forager will die.

Travel-home

The "travel-home" procedure (figure 8) is called as part of the "go" procedure, with foragers being set as the observer. After the procedure is called, a forager faces its home camp. Next, an if statement evaluates the proximity of the forager to its home camp: if the forager is located closer to the camp than the length of its foraging step, it moves to the camp's location. Furthermore, the forager's stored energy is transferred to the camp, and the "return-home" variable is set to "false". In the case that the camp started its migration process, indicated by the value of the "migrate-now" variable equalling "true", the forager dies.

In the case that the distance from the forager to the camp is greater than the value of the forage step, the forager moves towards its camp, the length of the step being set by the "forage-step" parameter. Finally, the energy cost of this step is subtracted from the forager's stored energy.

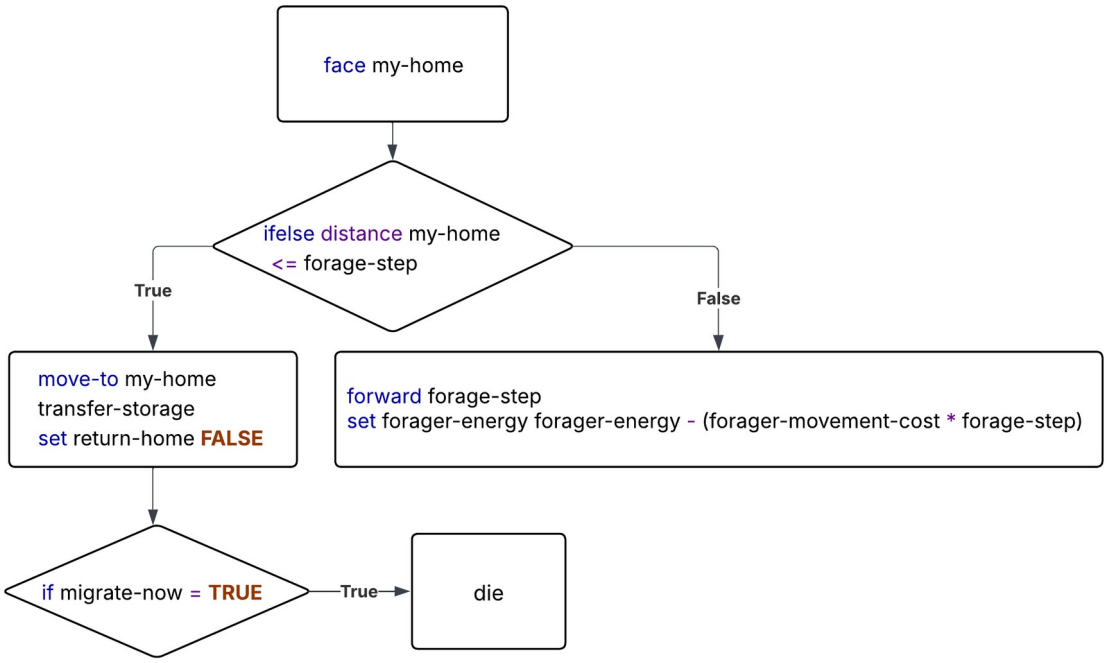

Figure 8: Flowchart of the "travel-home" procedure.

Transfer-storage

This procedure is called by "travel-home" procedure. It is run when a forager returns to its home camp. In the procedure, the forager transfers most of its stored energy to its home camp, keeping only the amount indicated by "init-forager-energy" parameter. This is done to ensure the forager has enough energy to embark on another foraging expedition.

Increase-population-check

"Increase-population-check" procedure is run as a part of the "go" procedure, with camps set as the observer. It evaluates whether the amount of stored energy in a camp surpasses the "population-growth-cost", and if a randomly generated floating point number is smaller than the "population-growth-chance". If that is the case the camp decreases its "camp-energy" by "population-growth-cost", and the value of the population variable of the camp is increased by one.

Update-display

Called at the end of the "go" procedure. Patches are set as the observer, and the procedure changes colour of each patch reflect the amount of available energy.

Regrow-patches

Called at the end of the "go" procedure. Patches are set as the observer. In the case that the "pempty" variable of the patch is set to "false", and the patch has less energy than the maximum possible energy value, the "penergy" variable of the patch is increased by "growth-rate".

Design concepts

Movement

Movement of foragers during foraging and camps during their migration process is modelled with random walks. The agents have an equal probability of moving onto any location within their movement radius, determined by the input parameters of the model.

Migration

Migration of camps is modelled according to the Patch-Choice model (Barton 2013). The average energy return rate of the environment is known to agents. Each forager tracks the amount of energy harvested from the last ten encountered patches, and calculates the mean encounter rate. Meanwhile, the camps track the mean encounter rate of their foragers. Once the mean encounter rate of a camp's foragers decreases below the mean average energy return rate of the environment, a camp begins the migration process.

Parametrization

All parameters which are used in this project are arbitrary. This is mainly due tot he limited scope of the project. After the model was developed, several parametrisation runs were conducted, using NetLogo's BehaviourSpace tool.

It was decided that the hypothesis of the study will be evaluated by changing the following two parameters: migration-step and forage step. The goal of those runs was to find a combination of the rest of the parameters with which it was possible to reach an equilibrium. Thus, it was decided that the following parameters will remain constant:

Parameter	Value
max-penergy	51
population-growth-cost	31
split-pop-pop-threshold	20
camp-number	1
pop-growth-chance	0.054
pop-energy-consumption	1
camp-movement-cost	1
forager-movement-cost	1
growth-rate	0.15
starting-population	28
init-forager-energy	19
resource-density	71
init-camp-energy	2000
split-pop-energy-threshold	2714
gather-rate	40
return-home-threshold	170
split-population-threshold	17

Table 4: Table with constant parameter values.

Results

Experiment 1 – Migration-step variation

During the exploratory part of the project, two experiments were conducted. In the first experiment, the forage-step parameter was kept constant (at the value of 1), while the value of migration-step varied. This experiment was conducted to understand how the success of foraging strategy depends on migration-step. The experiment was conducted with 5 different values for the migration-step parameter. There were 120 runs per parameter (600 runs in total). The stop conditions of each run were the death of all agents, or that the run reaches 20000 ticks. The experiment recorded the duration of each simulation run (table 5, figure 9).

Migration-step	10	15	20	25	30
Mean duration of runs (in ticks)	11680.891667	14901.191667	17203.725	16337.175	16238.741667

Table 5: Results of experiment 1.

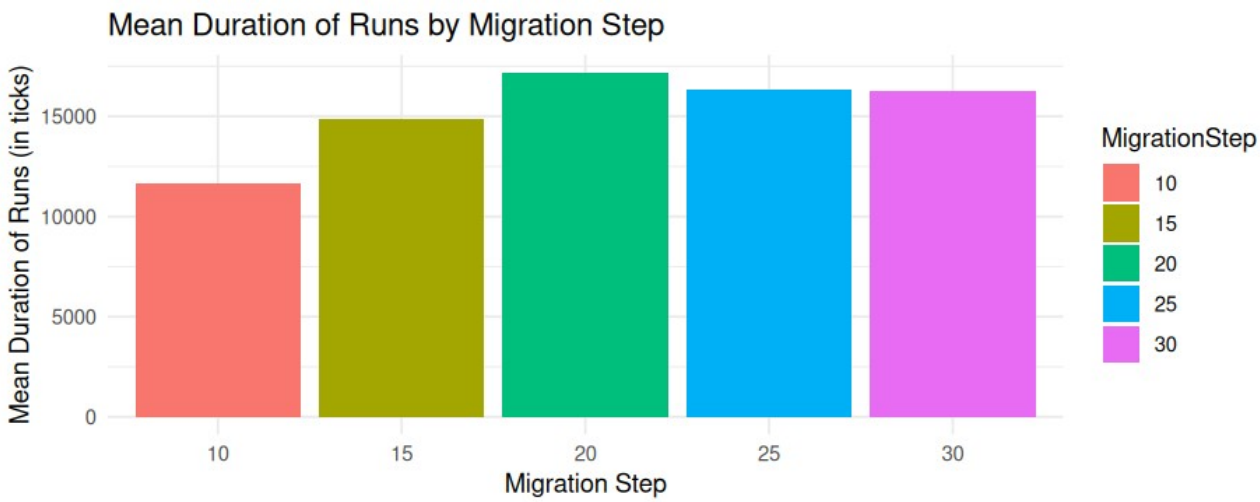

Figure 9: Bar chart visualising the results of experiment 1.

Experiment 2 – variation of both Migration-step and forage-step

It was then decided that two migration-step values from the previous experiment will be used for further analysis. Since the main premise of the paper is to examine difference between foraging behaviour with a higher value of migration-step and forage-step (SB) and a foraging behaviour with lower value of those two parameters (HP), it was decided that the values of migration-step will be 15 and 25.

However, experiment 2 also included varying the foraging-step parameter. The experiment was conducted with three different values of forage-step step parameter (1, 2 and 3). Each combination of variables was run one hundred times, with 600 runs in total (table 6, figure 10).

	Forage-step		
Migration-step	1	2	3
15	15158.7889	7184.43	2208.85
25	16676.29	4662.84	897.33
	Mean duration of runs (in ticks)		

Table 6: Results of experiment 2.

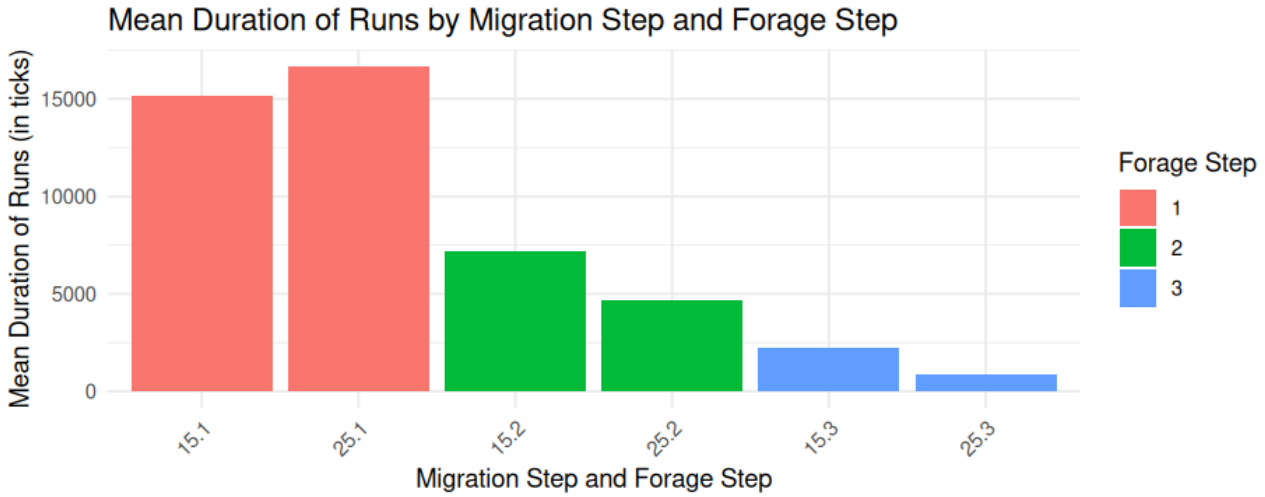

Figure 10: Bar chart visualising the results of experiment 2.

Discussion

The results of experiment 1 showed that migration-step parameter influences the success rate of the foraging strategy. Out of the five values with which the experiment was run, the lowest one (10) rendered the least successful foraging strategy. The foragers were most successful when the forage-step parameter was set to 20. However, parameter values higher than that (25, 30) showed less success.

Experiment 2 showed that forage-step also influences success of foragers. With the increase in the value of forage-step, the success rate of the foraging strategy decreases.

The premise of this paper was to observe difference between foraging behaviour with a higher value of migration-step and forage-step (SB) and a foraging behaviour with lower value of those two parameters (HP). However, this is not possible from the work done so far. As can be observed from experiment 2, the increase in forage-step leads to sudden decrease in the success of a foraging strategy, regardless of the value of migration-step. This is at least the case for the given parameter values.

On the other hand, experiment 1 showed that, given that the environment stays the same, change in migration-step influences the success of the foraging strategy. However, it is questionable whether this result sheds any light to the initial archaeological question. The increase in forager success rates with the increase of migration-step in experiment 1 seems to be easily explained: a if the migration-step is too short, the camp is not able to escape the area with depleted resources. The success rate then falls for migration-step values higher than 20 because foragers start expending too much energy on migration than is necessary to escape from the areas where they depleted the resources.

Furthermore, results could be also influenced by the model design. Both foraging behaviour and migration is modelled with random walks. However, it is unlikely that this accurately depicts Palaeolithic hunter-gatherer behaviour. Random walks are rather inefficient way to forage the environment. Therefore, this could be the reason for the sudden drop in success rates with the increase of forage-step.

Conclusion

The model developed for the purposes of this study simulates foraging behaviour of Palaeolithic hunter-gatherers. The goal of the study was to compare the success rates of two different foraging strategies in an identical environment: a higher value of migration-step and forage-step (presumably typical for Still Bay culture) and the one with lower value of those two parameters (representative of Howiesons Poort culture).

The two experiment runs conducted during this study failed to provide much meaningful insight in the matter. Possibly because of the errors in the parametrisation stage, the obtained results indicate that increase in foraging-step will lead to sudden decrease in success of foraging strategy, regardless of the migration-step. Still, experiment 1, in which migration-step was being varied, indicated that for some values of the migration-step parameter the foragers show more success. However, it is unclear whether this provides any useful information regarding the original archaeological question. Namely, it is possible that, with migration-step being too low, the camps are not able to escape the area with depleted resources.

Bibliography

Barton, C. Michael. 2013. ‘Patch Choice Model from Optimal Foraging Theory (Human Behavioral Ecology) v1.0.0’. *CoMSES Computational Model Library*, April 27. https://www.comses.net/codebases/2224/releases/1.0.0/.

McCall, Grant S., and Jonathan T. Thomas. 2012. ‘Still Bay and Howiesons Poort Foraging Strategies: Recent Research and Models of Culture Change’. *African Archaeological Review* 29 (1): 7–50. https://doi.org/10.1007/s10437-012-9107-y.